

Introducción

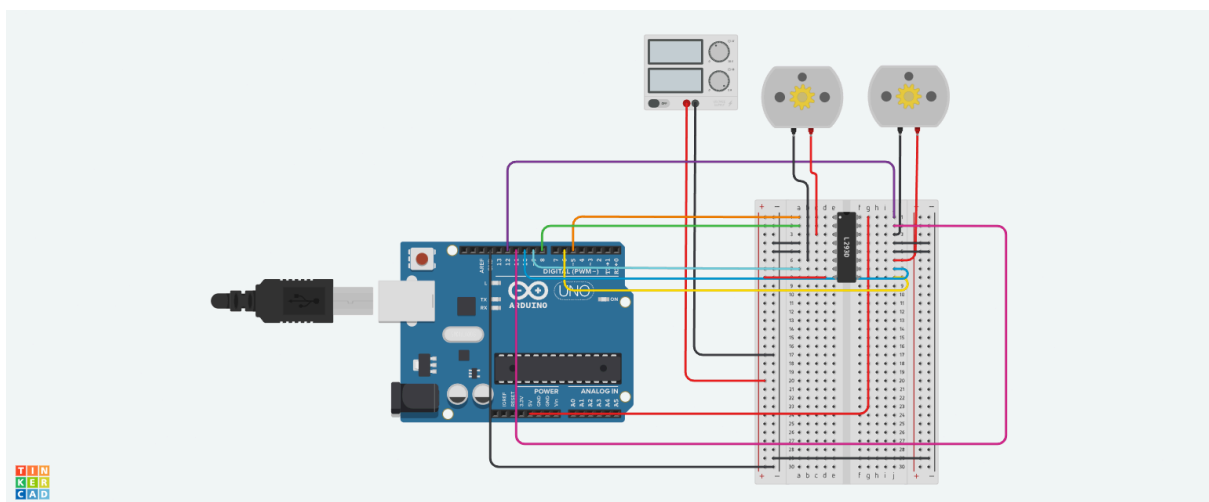
El objetivo de este proyecto fue crear un circuito controlado por un Arduino que pudiera controlar dos motores simultáneamente. Los dos motores deben de poder moverse ambos hacia enfrente al mismo tiempo, ambos hacia atrás al mismo tiempo y ambos a direcciones opuestas al mismo tiempo.

Metodología

Materiales:

- Arduino.
- Protoboard.
- 2 motores.
- Controlador de motor L293D.
- Cables jumper.
- Fuente de alimentación externa.

El Arduino fue utilizado para poder ejecutar el programa escrito dentro del circuito elaborado. Por otro lado, el protoboard era necesario para poder posicionar y conectar el resto de los componentes de manera sencilla. Dado el objetivo de este proyecto, el controlador L293D fue necesario para permitir el control de los dos motores de corriente directa, ya que permite controlar tanto su dirección como su velocidad mediante el Arduino. Se utilizó este controlador específicamente debido a que permite controlar los motores de manera independiente y facilita la conexión entre el Arduino y los motores, a diferencia del controlador utilizado inicialmente en el proyecto. Por otra parte, la fuente de alimentación externa fue necesaria para proporcionar la corriente requerida por los motores, evitando que el Arduino tuviera que suministrar directamente la energía necesaria para su funcionamiento, y para poder incrementar la velocidad de rotación de los motores.



En el nuevo circuito, el Arduino se conectó al controlador L293D para poder controlar los dos motores de corriente directa. Las entradas de control del L293D fueron conectadas a diferentes pines digitales del Arduino, permitiendo establecer la dirección de giro de cada motor y regular su velocidad mediante PWM. Las salidas del controlador fueron conectadas a los motores, mientras que la alimentación lógica del L293D se obtuvo del Arduino y la alimentación de los motores se proporcionó mediante una fuente externa. Finalmente, se conectaron las tierras del Arduino, el controlador y la fuente externa en común para establecer una referencia eléctrica compartida y permitir el correcto funcionamiento del circuito.

Código utilizado

```
1 // Motor delantero
2 int EN1 = 5;
3 int IN1 = 8;
4 int IN2 = 9;
5
6 // Motor trasero
7 int EN2 = 6;
8 int IN3 = 10;
9 int IN4 = 11;
10
11 void setup() {
12   pinMode(EN1, OUTPUT);
13   pinMode(IN1, OUTPUT);
14   pinMode(IN2, OUTPUT);
15
16   pinMode(EN2, OUTPUT);
17   pinMode(IN3, OUTPUT);
18   pinMode(IN4, OUTPUT);
19 }
20
21 void loop() {
22
23   // AVANZAR
24   digitalWrite(IN1, HIGH);
25   digitalWrite(IN2, LOW);
26
27   digitalWrite(IN3, HIGH);
28   digitalWrite(IN4, LOW);
29
30   analogWrite(EN1, 120);
31   analogWrite(EN2, 120);
32
33   delay(2000);
34
35   // ESPERA
36   analogWrite(EN1, 0);
37   analogWrite(EN2, 0);
38
39   delay(1000);
40
41   // RETROCEDER
42   digitalWrite(IN1, LOW);
43   digitalWrite(IN2, HIGH);
44
45   digitalWrite(IN3, LOW);
46   digitalWrite(IN4, HIGH);
47
48   analogWrite(EN1, 120);
49   analogWrite(EN2, 120);
50
51   delay(2000);
52
53   // ESPERA
54   analogWrite(EN1, 0);
55   analogWrite(EN2, 0);
56
57   delay(1000);
58
59   //DIRECCIÓN OPUESTA CON DIFERENTE VELOCIDAD
60   digitalWrite (IN1, HIGH);
61   digitalWrite (IN2,LOW);
62
63   digitalWrite (IN3,LOW);
64   digitalWrite (IN4, HIGH);
65
66   analogWrite(EN1,100);
67   analogWrite(EN2,120);
68
69   delay(2000);
70
71
72   // ESPERA
73   analogWrite(EN1, 0);
74   analogWrite(EN2, 0);
75
76   delay(1000);
77 }
```

Explicación del código

```
1 // Motor delantero
2 int EN1 = 5;
3 int IN1 = 8;
4 int IN2 = 9;
5
6 // Motor trasero
7 int EN2 = 6;
8 int IN3 = 10;
9 int IN4 = 11;
10
```

Primeramente, se van a definir como enteros las variables que se van a utilizar para cada motor. En el caso del programa usamos comentarios y espacios para poder facilitar la lectura del código. EN1 del pin 5 del arduino va ser utilizado para asignar el duty cycle de la señal PWM; esto último es el porcentaje de cada ciclo durante el cual el voltaje permanece en HIGH. Los valores IN1 y IN2, se van a utilizar para poder cambiar la dirección en la que rota el motor. Estas definiciones se repiten para el motor trasero; la variable EN2 con su pin 6 (PWM) e IN3, junto con IN4, utilizando los pines 10 y 11 respectivamente.

```
11 void setup() {  
12     pinMode(EN1, OUTPUT);  
13     pinMode(IN1, OUTPUT);  
14     pinMode(IN2, OUTPUT);  
15  
16     pinMode(EN2, OUTPUT);  
17     pinMode(IN3, OUTPUT);  
18     pinMode(IN4, OUTPUT);  
19 }
```

Dentro del void setup, se utilizan todos los pines de las variables como salidas.

```
21 void loop() {  
22  
23     // AVANZAR  
24     digitalWrite(IN1, HIGH);  
25     digitalWrite(IN2, LOW);  
26  
27     digitalWrite(IN3, HIGH);  
28     digitalWrite(IN4, LOW);  
29  
30     analogWrite(EN1, 120);  
31     analogWrite(EN2, 120);  
32  
33     delay(2000);
```

En la fase de avance se utiliza el digitalWrite para controlar la dirección de la corriente en el circuito. Al ser esta la fase de avance, se tomó como “avanzar” la rotación positiva de los motores, por lo que ambos motores rotan en la misma dirección. Adicionalmente, en las variables EN se asignó un valor de 120 mediante PWM, lo que corresponde aproximadamente a un duty cycle del 47%. Esto significa que, durante cada ciclo de la señal PWM, el voltaje permanece en HIGH aproximadamente el 47% del tiempo (para determinar el duty cycle se divide el valor del PWM entre 256). Por último se implementa un delay de 2000 milisegundos.

```
34  
35     // ESPERA  
36     analogWrite(EN1, 0);  
37     analogWrite(EN2, 0);  
38  
39     delay(1000);  
40
```

En esta fase, se asigna el duty cycle de ambos motores a 0 para que dejen de rotar, y se aplica un delay de 1000 milisegundos. Las fases de espera fueron creadas para mejorar la visualización de las pruebas individualmente.

```
41 // RETROCEDER
42 digitalWrite(IN1, LOW);
43 digitalWrite(IN2, HIGH);
44
45 digitalWrite(IN3, LOW);
46 digitalWrite(IN4, HIGH);
47
48 analogWrite(EN1, 120);
49 analogWrite(EN2, 120);
50
51 delay(2000);
52
```

En la fase de retroceder, se ejecuta esencialmente lo mismo que en la fase de avanzar pero con los valores de IN invertidos para cada motor. Esto causa que los motores giren en la dirección opuesta.

```
53 // ESPERA
54 analogWrite(EN1, 0);
55 analogWrite(EN2, 0);
56
57 delay(1000);
58
```

Segunda fase de espera.

```
59 //DIRECCIÓN OPUESTA CON DIFERENTE VELOCIDAD
60 digitalWrite (IN1, HIGH);
61 digitalWrite (IN2,LOW);
62
63 digitalWrite (IN3,LOW);
64 digitalWrite (IN4, HIGH);
65
66 analogWrite(EN1,100);
67 analogWrite(EN2,120);
68
69 delay(2000);
70
```

Dentro de los objetivos del proyecto estaba el de poder mover los motores en dirección opuesta y rotarlos a diferentes velocidades. Se utilizó el digitalWrite para poder asignar direcciones de rotación opuestas al motor frontal y trasero, y en el analogWrite se asignaron diferentes duty cycles para que los motores no rotaran a la misma razón por minuto; en esta prueba el motor trasero rota ligeramente más que el motor frontal.

```

71
72 // ESPERA
73 analogWrite(EN1, 0);
74 analogWrite(EN2, 0);
75
76 delay(1000);
77 }

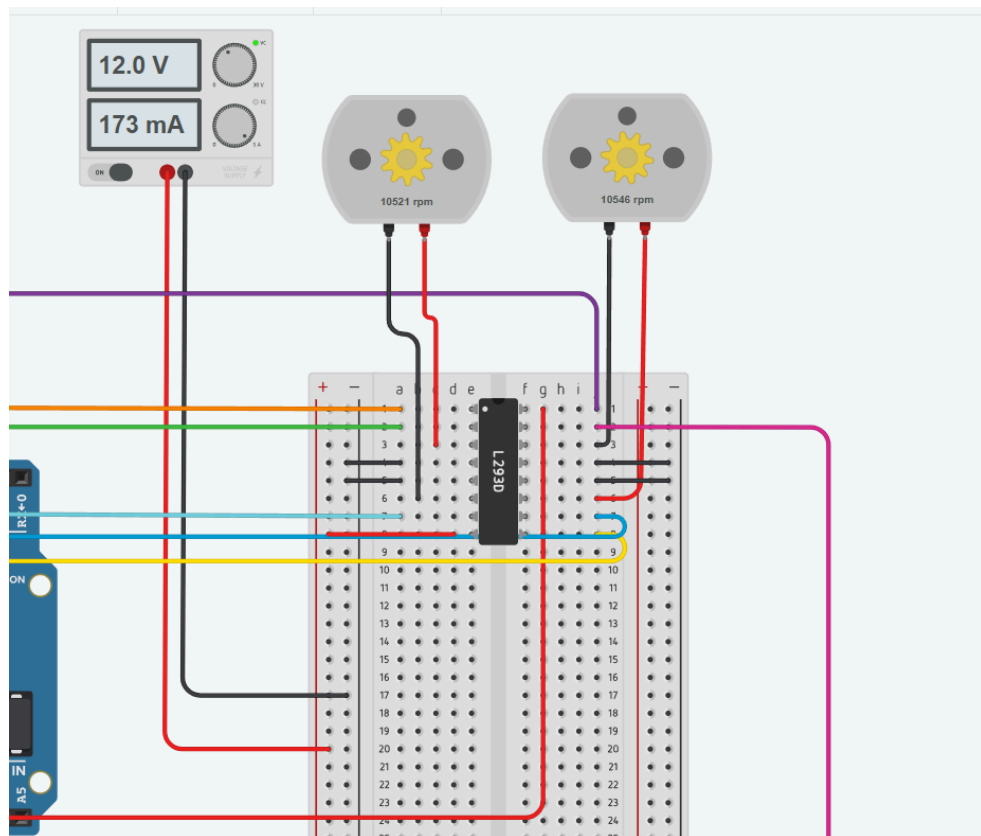
```

La última fase de espera antes de repetir el algoritmo.

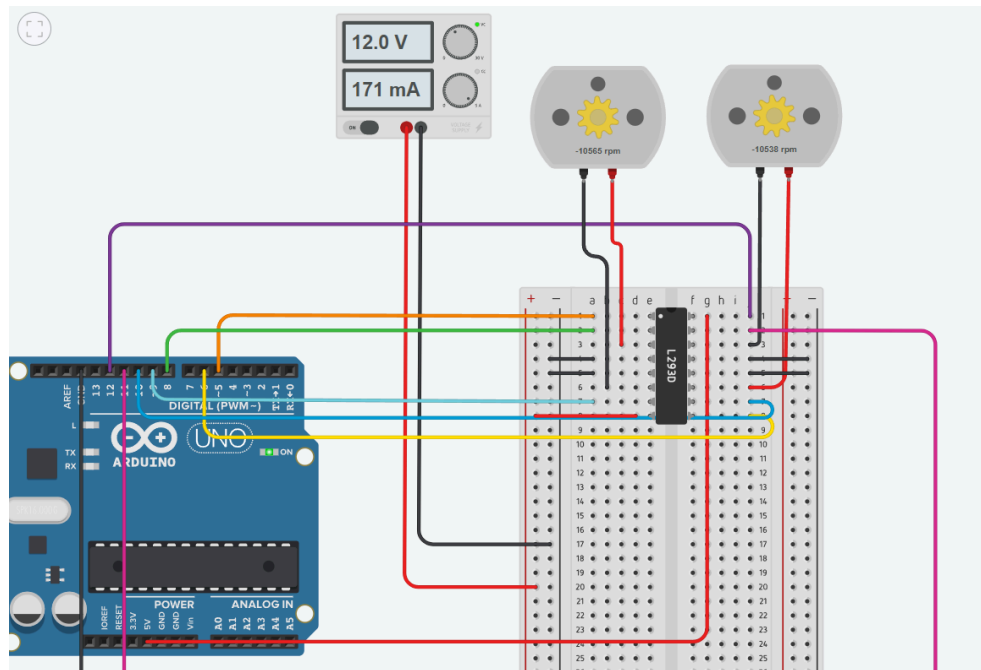
Resultados

Una vez el sistema conectado y el programa subido al Arduino, se probó su funcionamiento. Los resultados de este dieron lo siguiente:

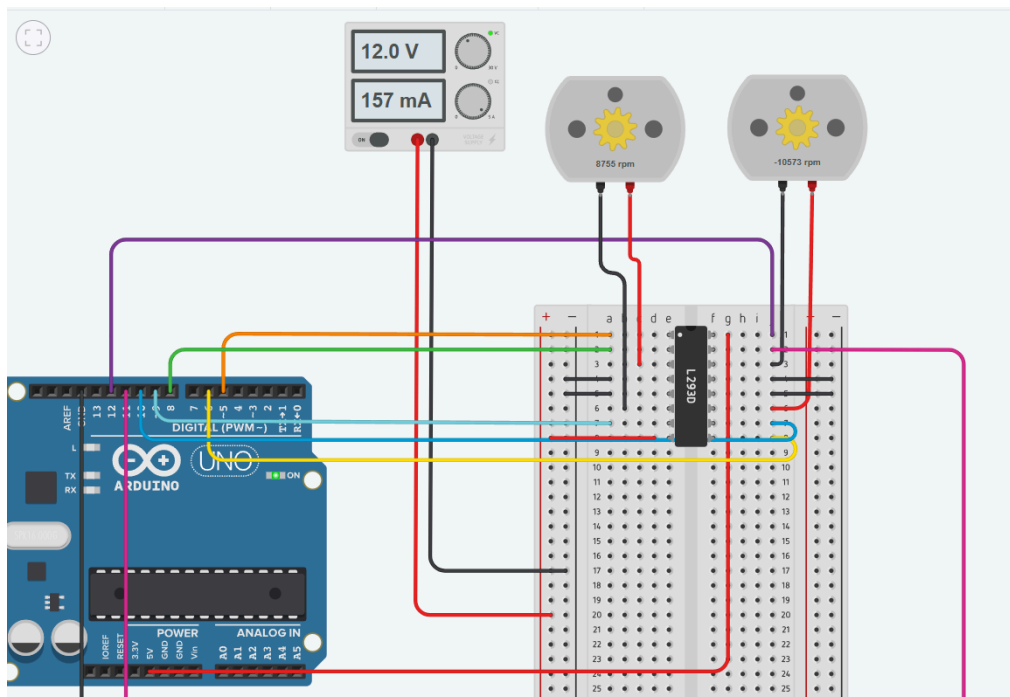
- La primera fase de prueba donde los motores giran a velocidades similares.



- Después de la primera fase de espera, los motores procedieron a probar el retroceso con velocidad igual en los pares de llantas.



- Después de otro delay, empieza la fase de prueba de par de llantas a dirección opuesta con velocidades diferentes.



- El circuito repite las fases de prueba hasta que el usuario lo detiene.

Conclusión

Los resultados obtenidos a partir del proyecto fueron los buscados. Los dos motores lograron funcionar correctamente de acuerdo con las especificaciones planteadas, permitiendo controlar tanto su dirección como su velocidad. Esto permitió comprender el

funcionamiento del controlador L293D y su importancia para establecer una conexión adecuada entre el Arduino, la fuente de alimentación externa y los motores.

Por otro lado, el control de las revoluciones por segundo fue un aspecto importante del proyecto, puesto que permitió observar cómo la velocidad de los motores puede ser modificada mediante el uso de PWM. A través de las diferentes pruebas realizadas, pudimos comprobar que al modificar la señal enviada al controlador también se modifica la velocidad de los motores, permitiendo que estos trabajaran a diferentes velocidades.

Por último, las pruebas realizadas con los motores en diferentes direcciones y velocidades nos permitieron comprender mejor la relación entre la programación y el funcionamiento físico del circuito. Esto ayudó a reforzar nuestros conocimientos sobre el control de motores y nos permitió identificar la importancia de realizar correctamente las conexiones del controlador para obtener el comportamiento esperado.